Internet of Things 31 (2025) 101565

Contents lists available at ScienceDirect

Internet of Things

journal homepage: www.elsevier.com/locate/iot

Research article

Bluetooth low energy indoor positioning: A fingerprinting neural network approach

Alberto Ferrero-López *, Antonio Javier Gallego, Miguel Angel Lozano

University Institute for Computing Research, University of Alicante, Carretera San Vicente del Raspeig s/n, 03690 Alicante, Spain

ARTICLE INFO

Keywords:
Indoor positioning
Bluetooth low energy (BLE)
Neural networks
Machine learning
Data filtering
Regression models
Classification models

ABSTRACT

This study explores the application of neural networks in indoor positioning using BLE (Bluetooth Low Energy) and the Fingerprinting location technique. The methodology involves two main phases: the capture and filtering process, where received BLE signals are smoothed and combined into fingerprint vectors, and the subsequent location prediction phase, which compares the position estimation from eight neural network designs and the classical trilateration method. We conduct a performance comparative analysis of each prediction method and study the optimal parameter values for the capturing and filtering processes. The research underscores the limitations of training metrics in reflecting real-world performance, emphasizing the importance of testing models on actual trajectories. Results indicate that regression neural networks outperform classification ones, and a complex dense neural network model proves most versatile and stable across testing scenarios. Our approach achieves a mean error of 1.9 meters, surpassing existing accuracies of 3.7 meters for trilateration and 3.1 meters for state-of-the-art neural network designs, thus holding promise for significantly improving indoor positioning accuracy with practical implications across various domains.

1. Introduction

1.1. Positioning techniques throughout history

Throughout history, humanity has constantly looked to answer fundamental questions such as "Where am I?" or "Where is my destination?" From ancient civilizations such as the Egyptians, Greeks, and Romans to the significant contributions of Picard and Cassini in large-scale triangulation, location methods have evolved remarkably [1–3].

The development of the Global Positioning System (GPS) in the late 20th century marked a major breakthrough in positioning technology. Initiated after the launch of Sputnik in 1957, GPS revolutionized global positioning through the use of radiofrequency-based triangulation with artificial satellites [3,4].

1.2. Challenges of GPS in indoor environments, alternative technologies

Despite its success in outdoor environments, GPS faces significant limitations indoors due to signal attenuation caused by walls, ceilings, and other obstacles. These factors negatively affect the accuracy of GPS positioning in enclosed spaces, making it unsuitable for precise indoor localization. These issues have encouraged the development of specialized technologies [5–8].

* Corresponding author.
E-mail addresses: afl13@alu.ua.es (A. Ferrero-López), jgallego@dlsi.ua.es (A.J. Gallego), malozano@ua.es (M.A. Lozano).

https://doi.org/10.1016/j.iot.2025.101565
Received 3 August 2024; Received in revised form 24 January 2025; Accepted 5 March 2025
Available online 12 March 2025
2542-6605/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

According to Hayward et al. [5], these technologies can be categorized into four principal groups, each offering unique approaches and advantages: inertial sensors, acoustic sensors, computer vision, and electromagnetic signals.

Within these four categories, inertial sensors are common and inexpensive components currently available in any mobile phone; however, the accumulation of reading errors provokes a long-term low accuracy. Acoustic sensors are also an inexpensive technology but their accuracy is highly dependent on the environment and the surrounding noise. Approaches based on computer vision reveal high accuracy, although this technology requires high computing resources and, in some cases, specific and expensive devices. Finally, the electromagnetic signal group offers an intermediate solution with acceptable accuracy; it does not require specific devices or high computing resources, can reuse existing environment resources (such as current WIFI signals), and is not affected long-term by an accumulation of errors.

Among electromagnetic signals, the Received Signal Strength Indicator (RSSI) stands out as one of the most utilized techniques. RSSI measurements, which are calculated as the difference between the transmitted and the received power, provide an indication of distance through signal strength variation as the device moves [9].

1.3. Fingerprinting using BLE RSSI signals

RSSI is used for basic indoor localization techniques such as trilateration, similar to GPS's global approach, as well as in more advanced methods like Fingerprinting. Fingerprinting localizes a device based on the readings from all surrounding sensors comprising two phases: the offline and the online phases. The offline phase executed on a preparatory stage requires the system to use the received sensors' values to create radio maps or fingerprints at known positions. The online phase executed during real-time positioning predicts the device's location by composing a radio map of the unknown position and comparing it with the stored fingerprints from the offline phase.

For fingerprinting, we can follow two approaches: probabilistic and neural network-based. The probabilistic-based approaches use mathematical methods and algorithms to find correspondences between the fingerprint vectors and estimate the position; for example, the calculation of the Wasserstein distance [10], or the use of Bayesian estimators [11]. By contrast, neural network-based systems initially run a training phase using the offline fingerprint data; during this phase, the neural network learns to predict the position from the fingerprint values. Both approaches' main troubles are related to the intrinsic challenges of electromagnetic signals, including attenuation, noise, and multipath interference, which are particularly common when employing Bluetooth Low Energy (BLE) signals [11].

Within the group of neural network-based approaches, the complexity of these systems relies on (i) properly cleaning and preparing the information to feed the network, (ii) defining a network structure able to capture the input's properties, and (iii) choosing the best training parameters. Most analyzed studies in this group fail to provide information on one or more of these critical elements [12–14]. Additionally, we found that some of these studies focus on the results and conclusions derived from *loss* and *accuracy* values resulting from the training phase, often lacking real-world tests. They provide information about the training results related to the offline fingerprint phase but fail to indicate how well the design performs in the online phase with an adequate metric. By themselves, *loss* and *accuracy* values may not reliably reflect a model's actual accuracy, especially given the signal distortion issues mentioned earlier. Without this real-world testing, it is difficult to measure the neural network's ability to predict a position from unseen fingerprint radio maps, which is the ultimate objective.

1.4. Our contributions

In this work, we propose a neural network-based methodology that integrates a capturing and filtering process to retrieve and clean BLE-received RSSI values, forming fingerprint vectors. This is combined with a neural network model configured to provide the position estimation using these fingerprint vectors as input.

Our study aims to analyze the optimal parameters of the proposed filtering function to minimize the challenges resulting from the use of the BLE signals, propose different neural network models, and evaluate their performance over actual trajectories in both noisy and noise-free conditions. We explore regression and classification methods, offering a comprehensive view of each approach's strengths and limitations. Finally, we determine if training *loss* and *accuracy* values accurately reflect the models' performance during live executions.

In summary, our main contributions are:

- A new capturing and filtering method designed to capture RSSI signals and create fingerprint vectors.
- A comprehensive analysis of the optimal parameters for capturing and filtering RSSI signals.
- Proposal, tuning, and evaluation of seven neural network architectures designed for indoor positioning using the fingerprint technique.
- A detailed comparison between our proposed models and existing state-of-the-art methods, tested with real trajectories under two different noise level conditions.
- Open-source release of the proposed capturing and filtering method, as well as the proposed network architectures and trained weights, available as publicly accessible Python libraries on GitHub.

This article presents a coherent overview of the research, beginning with related works (Section 2), followed by our methodology (Section 3), and the experimental setup (Section 4), which covers the dataset description, capture and filtering setup, and the indoor localization methods. Additionally, the document includes the results of the experiments (Section 5) and concludes with a synthesis of our key findings and contributions to the field (Section 7).

2. Related works

In this section, we review previous research directly related to our study. We organize these works into three distinct categories: probabilistic, neural networks, and trilateration approaches.

2.1. Probabilistic approach to fingerprinting

Daniş et al. [10] introduced a novel indoor positioning method known as affine Wasserstein histogram interpolation. Their approach employs histograms to compose the fingerprints and the Wasserstein distance to interpolate them as part of the update step in a Sequential Monte Carlo (SMC) particle filter, predicting the device position. In contrast, our approach does not rely on histogram creation or the use of SMC filtering, which simplifies the computation process by using neural networks for direct position estimation.

In their subsequent study [15], Daniş et al. improved their research by introducing an adaptive Monte Carlo filter designed to handle scenarios with non-uniform movement. Using this new method they achieved a median localization error of 2.22 m and a mean of 3.1 m. In our study, using the same dataset, we achieved a median localization error of 1.5 m and a mean error of 1.9 m, suggesting that our approach could offer better precision under similar conditions.

The work by Faragher et al. [11] is also relevant. It includes a deep analysis of the BLE technology, underlining its inherent electromagnetic challenges and proposing a capture signal process. The analysis presented in this work inspired our methodology phase of signal capture and filtering.

Xuyu Wang et al. [16] presented an interesting approach by proposing a probabilistic approximation over feature vectors or embeddings extracted from neural networks. In this work, the embeddings serve as radio maps while a probabilistic method determines the device's location. A key aspect of their approach is the use of Channel State Information (CSI) signals, which offer fine-grained data but require specific hardware with modified firmware for extraction, limiting its accessibility. Conversely, our method uses RSSI signals which can be collected using standard devices without specific software, making it easier to implement.

2.2. Fingerprinting with neural networks

Apiruk Puckdeevongs [12] proposed a neural network model to predict the position of a device using the indoor fingerprinting technique. This work includes a set of guidelines for designing neural network architectures that use fingerprints; however, it does not detail how the signals are captured or processed. Our study builds on this by comparing the neural model based on these guidelines against more complex architectures across scenarios with and without noise, demonstrating potential improvements in performance. Additionally, we expand on this by including a description of the signal capturing and filtering process employed to create the fingerprint vectors.

Ranimol Joseph et al. [13] employed neural networks for Wi-Fi RSSI signal-based positioning using the UJIIndoorLoc dataset [17]. This study utilizes an encoder and a classifier to predict the building and the floor where the device is located. Although interesting, this work lacks detailed information on the network setup. The evaluation is analyzed through the *loss* and *accuracy* training values but omits performance details from real tests. In contrast, our study includes an evaluation based on actual trajectories, measuring the error in predicting points not available during training phases.

Adege et al. [14] integrated a classification neural network for identifying the building floor and a regression neural network for device positioning. However, the study omits in-depth details about the signal capture and filtering process, the architecture of the neural networks, and how the proposed design manages missing values. Our work expands on this by providing a detailed description of the signal capture and filtering function, and by conducting tests both with and without missing values to evaluate the models' robustness under different conditions.

2.3. Positioning using trilateration with BLE RSSI signals

Kamil Szyc et al. [18] presented a comprehensive study in which four positioning algorithms—evolved from basic trilateration—are proposed. Our study adopts the window-based signal capture implementation from their work as part of our capture and filtering algorithm. Additionally, we evaluated the performance of the Total Signal Strength (TSS) filter used in their study, and compared the location error rates between the basic trilateration method and neural network architectures to evaluate potential improvements in accuracy.

3. Methodology

This section outlines the proposed methodology, visually exposed in Fig. 1 to provide a proper explanation. As the diagram shows, the methodology begins with BLE signals being broadcast from a beacon and captured by a set of sensors. These sensors communicate the readings to a window-based capture system designed to retain and smooth the most recent signals.

Once the window buffer accumulates a specific amount of collected signals, the captured data is filtered to reduce electromagnetic noise and combined into a tuple to compose the fingerprint data. This tuple is stored during the offline phase and directly utilized to feed the localization process during the online phase.

Following this, during the initial stage of the localization process, the data is preprocessed in a specific way depending on the selected localization method, which is subsequently executed to estimate the position.

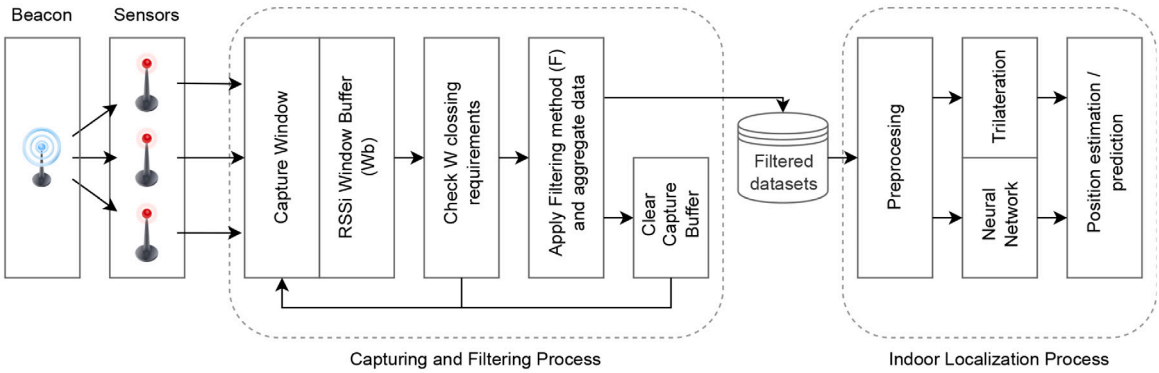

Fig. 1. Proposed methodology diagram. Our approach is based on two main phases: the capturing and filtering process, employing a window capture function, and the indoor localization process, which compares neural networks and trilateration.

3.1. Capturing and filtering BLE RSSI values

This section details the proposed process for capturing and processing RSSI values from the sensors, which is the initial step of our methodology. While our experiments primarily utilize the *Position-Annotated-BLE-RSSI-Dataset*, as outlined in Section 4.1, the process we describe is versatile and can be applied to other data sources, including real-time data capturing.

The capture and filtering process begins with the reception of RSSI signals, emitted from a beacon and captured by a set of $Sensors$. Each sensor individually transmits its RSSI readings to the time-based window capture system, which stores them in its buffer W_b. The length of the window is defined by two parameters given in seconds: $\omega_{\min}$ for the minimum length and $\omega_{\max}$ for the maximum. These parameters are determined experimentally, as detailed in Section 5.1; however, at the end of this section, the obtained values are anticipated as a reference.

This capturing process starts expanding the window when the first sample is received. The window cannot be closed until its length reaches the $\omega_{\min}$ value. If the window's length reaches the $\omega_{\max}$, it stops expanding and starts shifting, discarding the oldest samples.

Algorithm 1 formalizes this process. The algorithm's input consists of the window buffer (W_b) and three values: the sample's RSSI value ($rssi$), the reading's timestamp (ts), and the MAC address of the sensor that captured the sample (mac). In lines 2 and 3, the algorithm groups $rssi$, ts, and mac into a tuple and adds it to the buffer W_b. After that, in line 4, the algorithm removes the samples from W_b whose timestamps are older than $\omega_{\max}$. Next, it checks if the window can be closed in line 5; if so, it invokes the filtering function on the W_b content in line 6 and clears W_b to restart the process in line 7.

To close the window, a minimum number of valid sensors $S_{\text{valid_min}}$ is required. A sensor is considered valid only if it has accumulated a minimum number of measurements $S_{\text{reads_min}}$ in W_b.

The process responsible for determining whether the window can be closed is detailed in Algorithm 2. In lines 2–5, the algorithm checks if the timestamp of the first element in W_b is older than $\omega_{\min}$; if not, it returns $false$. Then, the algorithm counts the number of valid sensors in W_b. To achieve this, it groups the samples in W_b based on their origin MAC address in lines 6–8. After that, it counts the number of sensors whose amount of samples meets $S_{\text{reads_min}}$ in line 9. Finally, the algorithm returns $true$ if the number of valid sensors reaches $S_{\text{valid_min}}$; otherwise, it returns $false$.

The filtering phase starts once $S_{\text{valid_min}}$ is achieved (see Algorithm 3). Here, a filtering function f (more details in Section 4.2) consolidates the measurements into a single value per sensor, creating the fingerprint for the current position. Sensors failing to meet $S_{\text{reads_min}}$ receive a default value to mark them as empty.

The algorithm initializes an empty set at line 2 (F_v) to accumulate the filtered values from each sensor, composing the fingerprint. This process is described in lines 3–6, where a loop iterates over the $Sensors$ MAC list, extracting the stored values of each MAC in line 4 and assigning the fingerprint value in line 5. The fingerprint value depends on the number of stored samples; if it equals or exceeds $S_{\text{reads_min}}$, the filtering function f combines them to create the final value; otherwise, a default value of 100 is assigned (refer to Section 4.2 for more details).

The effectiveness of the capturing and filtering process depends on the fine-tuning of its parametrization: $S_{\text{reads_min}}$ as the minimum amount of measurements to validate a sensor, $S_{\text{valid_min}}$ as the minimum number of valid sensors required to close the capture window, f as the filtering function after window closing, and $\omega_{\min}$ and $\omega_{\max}$ to determine the window length.

These key parameters were defined experimentally by balancing positioning accuracy, number of samples, and frequency of readings. $S_{\text{reads_min}}$ was set to 2 after tests showed that this value offered the best balance between minimizing positioning error and ensuring enough sensor readings (see FST1 in Section 5.1). $S_{\text{valid_min}}$ was set to 12 because of its best accuracy in noise-free environments (without empty values allowed); for tests in noisy environments (with empty values allowed), 10 was selected because of its better balance between accuracy and number of samples (see FST2). For the filtering function f, the *Max* filter was chosen as it demonstrated better performance in reducing positioning error (see FST3). Finally and following the same balance objective, the window length parameters $\omega_{\min}$ and $\omega_{\max}$ were set to 0.5 and 2.5 s, respectively (see FST4 and FST5). The full experimental process is detailed in Section 5.1.

Algorithm 1 Accumulation of RSSI values

INPUT: $W_b, rssi, ts, mac$
$\mathbf{r} \leftarrow (rssi, ts, mac)$ ▷ Create a new tuple with the inputs
$W_b \leftarrow W_b \cup \mathbf{r}$ ▷ Add the new tuple to the buffer W_b
$W_b \leftarrow \{\mathbf{r} \in W_b \mid (\text{current_time} - r_{\text{ts}}) < \omega_{\max}\}$ ▷ Remove from W_b samples older than $\omega_{\max}$"
if WindowCloseCheck(W_b) **then**
  $F_v \leftarrow$ FilterValues(W_b) ▷ Function to filter and aggregate data
  $W_b \leftarrow \emptyset$ ▷ Reset Window Buffer
  return F_v
end if
return *false*

Algorithm 2 Checking if the capture window can be closed

function WindowCloseCheck(W_b)
  $(rssi, ts, mac) \leftarrow W_b^{(0)}$ ▷ Recover the first read from the buffer
  if $(\text{current_time} - ts) < \omega_{\min}$ **then**
    return false ▷ The window had not reached the $\omega_{\min}$ value yet
  end if
  for $m \in Sensors$ **do**
    $W^{(m)} \leftarrow \{\mathbf{r} \in W_b \mid r_{\text{mac}} = m\}$ ▷ Group the readings by MAC
  end for
  $S_{\text{valid}} \leftarrow \sum_{m \in Sensors} 1 \cdot (|W^{(m)}| \geq S_{\text{reads_min}})$ ▷ Count the number of valid sensors
  return $S_{\text{valid}} \geq S_{\text{valid_min}}$
end function

Algorithm 3 Filtering and aggregating data

function FilterValues(W_b)
  $F_v \leftarrow \emptyset$
  for $m \in Sensors$ **do**
    $W^{(m)} \leftarrow \{\mathbf{r} \in W_b \mid r_{\text{mac}} = m\}$ ▷ Group the readings by MAC
    $F_v^{(m)} \leftarrow \begin{cases} f(W^{(m)}) & \text{if } |W^{(m)}| \geq S_{\text{reads_min}} \\ 100 & \text{otherwise} \end{cases}$ ▷ Filter sensors with insufficient readings
  end for
  return F_v
end function

3.2. Indoor localization models and algorithms

In the second phase of the proposal (see Fig. 1), the resultant output of the capturing and filtering process feeds the indoor localization process. For this, we propose different approaches based on neural networks: regression models, designed to predict the precise x and y coordinates, and classification models, which segment the space into a set of possible locations. Additionally, we include an overview of the trilateration method for comparison with the proposed models.

Further details on neural networks' hyperparameters and training specifics are provided in Section 4.4. Additionally, the specific configuration of each network is summarized in Table 1. This table is structured to visually represent the architecture of each neural network, displaying the layers from left to right, similar to a graphical illustration.

3.2.1. Baseline neural network

Puckdeevongs [12] proposed a list of recommendations for the implementation of an indoor localization process based on neural networks. These guidelines suggest using a single hidden layer with a neuron count equating to two-thirds of the input layer size multiplied by the output layer size, ensuring it does not exceed twice the size of the input layer. Following these rules, we implemented our baseline architecture M1, details of which are found in Section 4.4.

3.2.2. Regression neural networks

Building on the top of the M1 design, we proposed a series of modifications to investigate how to improve the effectiveness of this baseline. These are the details of the proposed regression models:

Table 1
Neural networks' configurations. Notation: *D(n)* represents a dense or fully-connected layer with *n* neurons, *Concat* indicates a concatenation layer, *Cnv(f, s)* corresponds to a one-dimensional convolutional layer with *f* filters and a kernel size of *s*, and *MaxPool(s)* represents a max-pooling layer with a size of *s*.

Model	Input	Network architecture							
		Layer 1	Layer 2	Layer 3	Layer 4	Layer 5	Layer 6	Layer 7	Layer 8
M1	RSSI values	D(10)							
M2	RSSI values	D(1024)	D(128)	D(16)					
M3	RSSI values Empty values mask	Concat	D(1024)	D(128)					
M4	RSSI values Empty values mask	D(64) D(16)	D(512) D(512)	D(32) D(512)	Concat	D(16)			
M5	RSSI values	Cnv(64 × 7)	Cnv(256 × 7) MaxPool(6)	D(128)	D(512)	D(1024)			
M6	RSSI values	D(16)	D(32)	D(81)					
M7	RSSI values	D(16)	D(32)	D(42)					
M8(3 × 3)	RSSI values	D(512)	D(512)	D(512)	D(64)	D(256)	D(2048)	D(128)	D(256)
M8(9 × 9)	M8(3 × 3) Output	D(32)	D(128)	D(128)	D(64)				

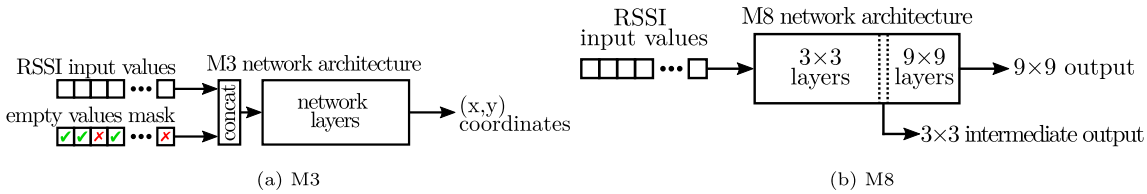

Fig. 2. Diagrams of two of the proposed architectures: a) M3 performs regression to estimate the x and y position coordinates and incorporates an additional input mask to indicate the validity of the input sensors; and b) M8, based on position classification, which is organized into two stages. The first part performs a coarse classification on a 3 × 3 grid, while the second part refines the position within a 9 × 9 grid.

- **M2 - Dense neural network:** Configuration with additional hidden layers and a higher number of neurons than M1. This structure aims to improve the model's ability to capture relationships and patterns in the data.
- **M3 - Dense neural network with an additional input mask:** It employs two inputs, the RSSI signals and a boolean mask indicating the sensors' validity as follows: $mask \leftarrow \bigcup_{i=1}^{|Sensors|} m_i$, where $m_i = (value_i \neq 100)$. This model combines both inputs in the initial step and then processes the resultant information (see Fig. 2(a)).
- **M4 – Dense neural network with an additional input mask (variant):** This model is built on M3's concept, improving it by first processing each input separately to extract features before combining them (see Table 1).
- **M5 - 1-Dimensional Convolutional Neural Network:** This model merges 1D convolutional layers with dense layers to capture and process the sequential input information.

3.2.3. Classification neural networks

In this approach, we explore different neural networks configurations using classification outputs to segment the positioning space into specific categories. Below, we present the details of the proposed models:

- **M6 - 9 × 9 Grid:** This design stands out by dividing the space into a 9 × 9 grid, resulting in a total of 81 cells or categories. This grid size choice aligns with the capture positions during the offline stage of the main dataset used in this study, which is better explained in Section 4.1.
- **M7 - 7 × 6 Grid:** The objective of this design is to provide an acceptable error margin by dividing the space into cells that represent an area of 3 square meters. The decision for these dimensions is based on the exploration of the feasibility of increasing the error margin in order to reduce the number of categories and, consequently, the likelihood of error.
- **M8 - Double Resolution Grid:** This design combines two neural networks. The first one, named M8(3 × 3), operates using a 3 × 3 grid for predicting an initial classification of the position. Subsequently, the second network, named M8(9 × 9), refines this positioning with a 9 × 9 grid (see Fig. 2(b)).

3.2.4. Trilateration

We included the trilateration method as a benchmark to compare its performance against our proposals. Trilateration is a classic positioning technique that relies on the measurement of signal strength to deduce a device's location. It operates by determining the distance between a device and several known reference points, creating circles around each as a representation of possible locations. The device's location is then estimated by the intersection of at least three of these circles.

As the input data for our system are RSSI signals, we needed to convert these signals into distances. This conversion was achieved using the following equation:

$$d = 10^{\frac{A_1 - RSSI}{10 \cdot A_2}} \tag{1}$$

where A_1 and A_2 are constants unique to each sensor. We determined these constants using a least squares reduction approach, as outlined in [18]. It is crucial to acknowledge that using this logarithmic formula assumes omitting environmental factors that can affect the Bluetooth signals, causing errors in the final estimations as explained in [19].

With these values determined, we proceeded with the trilateration process: For each point to be predicted, we selected the three (or four, if feasible) closest sensors based on their RSSI values. Employing Eq. (1), we converted these RSSI values into distances and then executed the trilateration algorithm.

4. Experimental setup

This section presents the configuration of our experimental approach. It includes the description of the dataset, the parameters used for the capture, filtering, and preprocessing steps, as well as the details about implementation, evaluation and training procedures.

4.1. Dataset overview

For the evaluation of our proposal, we resort to the *Position-Annotated-BLE-RSSI-Dataset* [20]. This dataset, designed for high-precision positioning with BLE RSSI data [21], has been integral to various indoor localization studies [10,15].

Data was gathered in a 20.66 × 17.64 m room with 12 BLE sensors and a BLE beacon. Accurate positions were estimated using Aruco visual tags, recorded via two GoPro Hero 4 cameras, with an error margin of around 0.05 m. Video analysis provided precise beacon positioning.

The dataset contains two key parts:

- **Samples at known room points (offline phase):** Captured at two sets of points, a 9 × 9 grid and a 5 × 9 grid. Each set includes sensor and position-specific files with raw received captures, containing timestamps, MAC addresses, and RSSI values.
- **Samples from live trajectories (online phase):** This includes captures from multiple trajectories within the room. Each trajectory file contains raw signals from each sensor, timestamps, MAC addresses, RSSI values, real positions on the X, Y, Z axes, and the 3 × 3 visual orientation matrix.

Following the guidelines of the paper that introduced this dataset [20], we rely on the 9 × 9 grid from the offline phase for model training, as it provides more comprehensive information, and use the online phase for evaluation. Also note that the training set was split in two, leaving 80% of the data for training and 20% for validation.

For each reference point, the dataset provides RSSI readings recorded continuously over a duration of 1800 s. Starting from the first available timestamp, our sliding time window processes this temporal range to construct fingerprints, dynamically aggregating data according to the parameters $\omega_{\min}$ and $\omega_{\max}$. This approach ensures that all readings are processed while simulating real-time conditions.

4.2. Capturing and filtering

The filtering phase starts once the capture window is closed, combining measurements into a single value for each sensor by applying a filtering function f. For this function, different options can be considered, including: mean, median, and extreme values, adhering to the methodologies outlined in [11]. Additionally, our study introduces TSS filtering, inspired by [18], which calculates a weighted sum of RSSI intensities, as represented by the given formula:

$$TSS = \sum_{i=1}^{n} 10^{\frac{RSSI_i}{10}} \tag{2}$$

For sensors that fail to meet the validity criteria $S_{\text{reads_min}}$, a default value of *100* is assigned. This approach, inspired by previous studies such as [13,22], is employed to represent unidentifiable or null measurements.

The capturing and filtering process has been implemented as a Python library to facilitate reuse in other projects. It has been made available as an open-source GitHub repository.[1]

[1] Capturing and filtering algorithm, implemented as a Python library: https://github.com/bertoferrero-research/rssi_capturing_filtering_library.

Table 2
Hyperparameters used for training neural networks with and without empty values.

	Model	Optimizer	Learning rate		Model	Optimizer	Learning rate
Empty values	M1	SGD	0.1	No empty values	M1	SGD	0.1
	M2	Adam	0.0001		M2	Adam	0.001
	M3	Adam	0.0001		M3	Adam	0.001
	M4	Adam	0.001		M4	Adam	0.001
	M5	Adam	0.001		M5	Adam	0.001
	M6	SGD	0.1		M6	Adam	0.001
	M7	SGD	0.1		M7	Adam	0.001
	M8	Adam	0.0001		M8	Adam	0.0001

4.3. Fingerprint preprocessing

The capturing and filtering process takes raw RSSI signals and transforms them into fingerprint vectors, which must be preprocessed before being used as input for the neural networks.

To normalize the RSSI values for the models' input we applied standard normalization, i.e., $(x - \mu)/\sigma$, where μ is the sample mean and σ denotes the standard deviation.

For position coordinates along the X and Y axes, we applied two approaches depending on the model type. In regression models, we applied rescaling (min–max normalization), considering the dataset's minimum and maximum values. For the X axis, the scaler range included values from 0 to 20.66 m, and for the Y axis, from 0 to 17.64 m. In the case of classification models, we transform the x and y coordinates into a numerical identifier corresponding to a specific cell (in the gridded scenario described previously). To recover the original position coordinates from these cell identifiers, we calculate the center position of the cell.

4.4. Hyperparameters and neural networks' configuration

This section describes the hyperparameters and training details of the neural networks proposed in our study.

Initially, we conducted a study to determine the hyperparameters of each architecture, excluding M1. For this, we utilized AutoML tools. Specifically, we employed AutoKeras [23] for models M2 to M7, and KerasTuner [24] for M8. The decision to use KerasTuner for M8 was motivated by the need to tune hyperparameters that AutoKeras does not support, such as the weighting of each submodel's contribution to the overall error function.

In the first phase, these AutoML tools were given the autonomy to automatically select any hyperparameter while each model was trained. In a second phase, we trained them again retaining the hyperparameters related to the neural network's structure and setting the AutoML tools to fine-tune the learning rate and select the optimizer exclusively.

We trained all models for up to 1000 epochs with a batch size of 256. For regression models (M1–M5), we used Mean Squared Error for loss and the Linear activation function as output. For classification models (M6–M8), Categorical Cross-entropy was used as the loss function, and Softmax as the activation for the model's output. These loss and activation functions were selected because they are the standard choices for regression and classification networks, respectively.

To prevent overfitting, we implemented early stopping callbacks. For models M1–M7, a single callback monitored *loss* on the validation set, with δ=0.0001 and patience of 10 epochs. Model M8 had a more complex setup with three callbacks: two for the neural network's outputs and another for the overall *loss*.

Table 2 shows a comprehensive overview of the choices eventually made for optimizer and learning rates of each model.

For more implementation details, the code employed for the definition of hyperparameters and training phases is available at its GitHub repository.[2] Note that, for the implementation of the trilateration we used the *easy-trilateration* library,[3] which relies on a least squares approximation.

4.5. Evaluation

To assess the precision of the compared models, we used the Euclidean distance for comparing the actual coordinates $(x_{\text{real}}, y_{\text{real}})$ with the predicted ones $(x_{\text{predicted}}, y_{\text{predicted}})$. This measurement effectively quantifies the deviation in meters between the predicted and actual values, thereby providing a clear indicator of the accuracy of our models when predicting real-world distances.

The deviations represent the error distribution of each test, which is evaluated by calculating its minimum, maximum, percentiles, average, and median values. Subsequently, these sets are visually exposed and compared by plotting box-and-whiskers charts.

5. Results

This section presents the results of the experiments, starting with the evaluation of the hyperparameters of the capture and filtering process, and then moving on to compare the different proposed methods.

[2] Models' definitions and trainers are implemented as a Python library: https://github.com/bertoferrero-research/models_fingerprint_positioning.
[3] *Easy-trilateration* a trilateration library for python: https://github.com/agusalex/easy-trilateration.

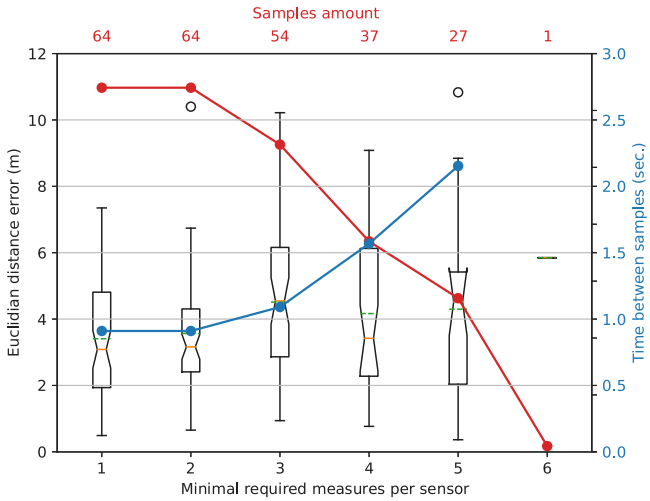

Fig. 3. Results of Filtering Settings Test 1 (FST1): Evaluation of the performance of $S_{\text{reads_min}}$ parameter which determines the minimum number of samples needed to consider a sensor as valid. The graph illustrates the distribution of positioning error (black), the amount of samples (red), and the average time in seconds between samples (blue).

5.1. Hyperparameters evaluation

As detailed in Section 3.1, the capturing and filtering algorithm depends on the precise adjustment of five key parameters. To optimize these parameters, we conducted a series of tests, each focused on balancing three metrics: Euclidian distance error for the real trajectory, quantity of samples remaining after filtering, and sample frequency.

We named these tests as "Filtering Settings Tests", using the contraction FSTn, where n denotes the test number, to refer individually to each one as follows: FST1 determines the minimum required number of measurements to consider a sensor as valid ($S_{\text{reads_min}}$), FST2 establishes the minimum number of valid sensors ($S_{\text{valid_min}}$), FST3 specifies the filtering function (f), FST4 sets the maximum window size (ω_{max}), and FST5 defines the minimum window size (ω_{min}).

These experiments were conducted using the M1 model, due to its simplicity and its restricted design. The executions of these tests require an initial establishment of parameters not yet optimized. Drawing from Faragher's work [11], we initially set ω_{min} to 0.5 s and ω_{max} to 2 s, selecting *Median* for f. The initial value of $S_{\text{valid_min}}$ is set to 3, aligning with the basic requirements for trilateration. And for $S_{\text{reads_min}}$ we did not set an initial value, intending to define it during the first test.

In each test, we established a range of values to be evaluated. For every value within this range, we carried out the capture and filtering process. Below we present the results for each of these experiments:

- **FST1.** We evaluated values of $S_{\text{reads_min}}$ ranging from 1 to 6. As shown in Fig. 3, the positioning error increases when fewer samples are used and when intervals between readings are longer. Among the tested values, 1 and 2 resulted in the lowest errors while maintaining a higher number of samples. Although the results are quite similar, a value of 2 produces a slightly lower maximum error. Additionally, this value ensures a minimum number of readings per sensor, enabling the filtering process to function effectively. Therefore, $S_{\text{reads_min}} = 2$ was selected as the optimal value.
- **FST2.** For this experiment, we assessed values of $S_{\text{valid_min}}$ ranging from 1 to 12. The results, shown in Fig. 4, reveal that positioning error remained relatively stable up to the value of 8, with a decline observed from 9 onwards, accompanied by fewer samples and slightly longer intervals between them. The most effective value was $S_{\text{valid_min}} = 12$, corresponding to the total number of sensors in the dataset; this value reduced the number of samples by approximately 34%, requiring no sensors without values. For tests allowing empty values, we chose $S_{\text{valid_min}} = 10$, which reduced the number of samples by about 11%.
- **FST3.** We evaluated different sensor filtering functions, including *max*, *min*, *mean*, *median*, and *TSS*. Based on the results observed in Fig. 5, the *max* filter showed slightly better performance compared to the others.
- **FST4.** We examined various maximum window sizes for the capturing step, ranging from 1 to 3 s. The results, illustrated in Fig. 6, suggest that the optimal window size in this case is $\omega_{\text{max}} = 2.5$ s.
- **FST5.** In this test, we assessed minimum window sizes for the capturing step, ranging from 0.25 to 1.75 s. The optimal value, as shown in Fig. 7, is obtained for ω_{min}=1.75 s. However, to avoid a drastic reduction in sample size (already trimmed in FST2), we opted to set this parameter to $\omega_{\text{min}} = 0.5$ s, which offered similar performance.

After conducting these tests, we found the following optimal values for our experiments. We defined ω_{min} at 0.5 s, ω_{max} at 2.5 s, $S_{\text{reads_min}}$ was set to be 2, ensuring each sensor contributes at least two readings in each window, and for f, we chose *Max* due to

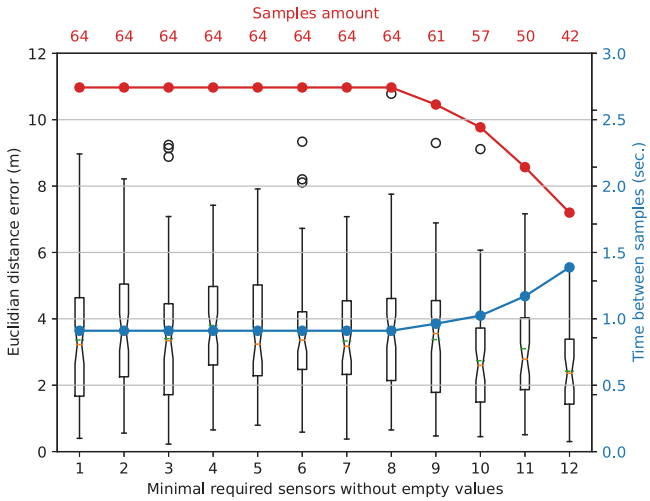

Fig. 4. Results of Filtering Settings Test 2 (FST2): Evaluation of the performance of $S_{\text{valid_min}}$ parameter, which determines the minimum number of valid sensors needed to close the capture window. The graph illustrates the distribution of positioning error (black), the amount of samples (red), and the average time in seconds between samples (blue).

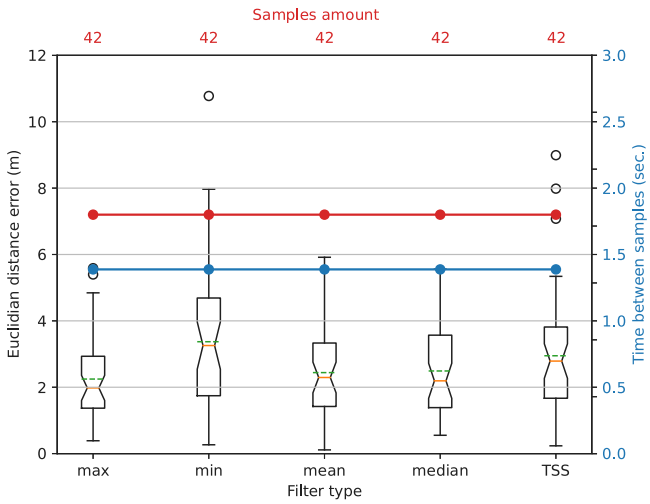

Fig. 5. Results of Filtering Settings Test 3 (FST3): Evaluation of the performance of f parameter, which determines the filtering type to apply to the reads captured from the same sensor to smooth errors and combine the reads in a single result. The graph illustrates the distribution of positioning error (black), the amount of samples (red), and the average time in seconds between samples (blue).

its effectiveness in data filtering. Additionally, we selected two values for $S_{\text{valid_min}}$, allowing for the creation of a filtered dataset with and without empty values in the RSSI signals. These values are 12 to ensure no empty values and 10 to permit them.

Having identified the optimal values, we then applied these parameters to the samples extracted from the dataset outlined in Section 4.1. To assess the impact of empty values on the performance of each neural network model, we generated two distinct filtered datasets by applying the two optimal values identified in FST2. Consequently, we created the *Filtered Dataset With Empty Values* and the *Filtered Dataset Without Empty Values*.

5.2. Neural network models evaluation

In this section, we present the evaluation of our proposed models. The results are divided into two groups based on the filtered dataset employed. Each subsection includes a table detailing loss, accuracy, and Euclidean distance errors. Furthermore, we provide box-and-whisker plots to illustrate error distribution and prediction accuracy.

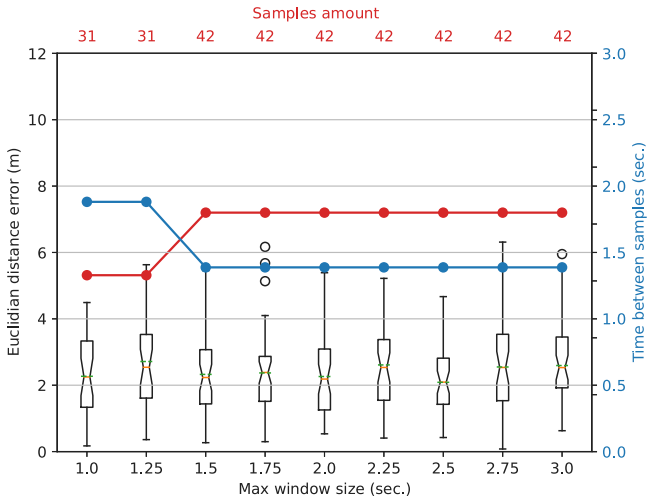

Fig. 6. Results of Filtering Settings Test 4 (FST4): Evaluation of the performance of ω_{max} parameter which defines the maximal length of the capture window. The graph illustrates the distribution of positioning error (black), the amount of samples (red), and the average time in seconds between samples (blue).

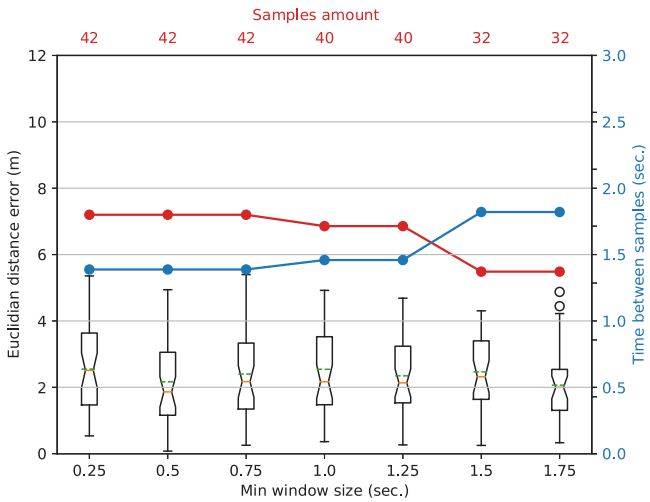

Fig. 7. Results of Filtering Settings Test 5 (FST5): Evaluation of the performance of ω_{min} parameter which defines the minimal length of the capture window. The graph illustrates the distribution of positioning error (black), the amount of samples (red), and the average time in seconds between samples (blue).

5.2.1. Filtered dataset with empty values

Table 3 presents the results obtained during the training and testing phases for each model, including the trilateration method. These results are based on the filtered dataset that allows empty RSSI values, i.e., when S_{valid_min}=10. Additionally, Fig. 8 shows the prediction error distribution for each model, along with the trilateration technique for comparison purposes.

These figures also show the best mean and median results obtained by the state-of-the-art method proposed by Danış et al. [15] using the same dataset. The method described in that study is based on an adaptive sequential Monte Carlo filter specifically designed to handle scenarios with non-uniform movement.

The examination of the data reveals that all proposed models improved the performance obtained by trilateration (T1) and the baseline model (M1). The state-of-the-art also obtained better results than T1 and M1; however, it was outperformed by the models M2, M3, M4, M5, and M7, and partially outperformed by M8.

The top-performing designs, characterized by lower mean and median errors as well as a narrower error distribution, are the models that include an auxiliary input where a mask is applied to indicate empty value entries (M3 and M4). Among these two, M4 has shown slightly better performance.

We observe a performance difference by comparing regression models (M1–M5) and classification models (M6–M8). The classification models exhibit slightly lower precision and a larger error distribution, which is especially noticeable in the case of M6. This difference may be attributed to the fact that errors in classification models result in larger distance errors, as selecting the wrong cell leads to a more substantial deviation.

Table 3
Euclidean distance error (in meters) for each model during the testing phase, using the filtered dataset with empty values. The metrics include the minimum, maximum, mean, and quartiles (25%, 50%, and 75%) of the error. This table also includes the mean and median best results from [15] using real RSSI values, as well as trilateration.

	Model	Testing results - Euclidian distance error (m)					
		Min	Max	Mean	25%	50%	75%
State of the art	Tri.	0.4801	11.1294	3.7252	2.1561	3.0720	4.9686
	[15]	–	–	3.064	–	2.223	–
	M1 [12]	0.9588	6.9745	3.5925	1.9589	3.5146	4.6506
Our proposals	M2	0.3524	7.1939	2.0368	**0.8275**	1.8199	2.6617
	M3	0.3001	**6.0084**	2.1760	1.3477	1.8099	2.5129
	M4	0.3346	6.6635	**1.8957**	1.0157	**1.5521**	**2.4376**
	M5	**0.0746**	10.4351	2.3199	1.2394	2.2031	2.8617
	M6	0.3043	12.1579	3.5502	1.8904	2.9658	4.3254
	M7	1.1569	8.6415	2.5391	1.4616	2.0487	2.8507
	M8	0.2884	7.2817	2.5579	1.6574	2.4521	3.3646

Table 4
Euclidean distance error (in meters) for each model during the testing phase, using the filtered dataset without empty values. The metrics include the minimum, maximum, mean, and quartiles (25%, 50%, and 75%) of the error. This table also includes the mean and median best results from [15] using real RSSI values, as well as trilateration.

	Model	Testing results - Euclidian distance error (m)					
		Min	Max	Mean	25%	50%	75%
State of the art	Tri.	0.4573	8.4107	3.3583	1.8876	3.0278	4.1647
	[15]	–	–	3.064	–	2.223	–
	M1 [12]	0.2713	**5.3440**	2.4161	1.6272	2.2874	2.9098
Our proposals	M2	0.2318	8.3649	**2.0559**	**0.9133**	**1.7745**	2.7371
	M3	0.4095	6.0506	2.2633	1.4240	2.1490	3.0611
	M4	0.2770	8.2601	2.1137	0.9279	1.9828	**2.5777**
	M5	0.2305	7.1876	2.0766	1.0047	2.0344	2.9631
	M6	**0.1770**	7.4779	2.7193	1.4660	2.5135	3.3330
	M7	1.1959	7.6961	2.7920	1.5811	2.2274	3.4713
	M8	**0.1770**	12.1579	2.8570	1.4660	2.4220	3.3670

Table 5
Accuracy results during the testing phase for classification models (M6–M8). The table shows accuracy values for both datasets: with and without empty values.

Model	Testing results - Accuracy	
	Empty values	No empty values
M6	0.1930	0.2142
M7	0.3860	0.3571
M8	0.7719/0.2281	0.7142/0.1666

When comparing M7 with M6 and M8, it becomes evident that employing a reduced number of output classes has led to enhanced performance, even though this decision implied the augmentation of cell sizes and the margin of error. M7 shows median and maximal values lower than both M6 and M8, primarily due to its lower rate of failures in predicting the correct cell.

Table 5 exposes the accuracy trade-off of these classification models. It is important to note that, even though the accuracy results show a low rate, the results based on the Euclidean distance error (Tables 3 and 4) provide a good accuracy rate in contrast. This discrepancy arises because misclassifications are treated equally regardless of the actual distance between the predicted and correct cells. For instance, misclassifying a nearby cell is considered as incorrect as misclassifying a cell at the opposite end of the room. Hence, accuracy is not an ideal metric for measuring the efficiency of classification neural models in fingerprinting.

Finally, loss and accuracy metrics from training or testing phases do not necessarily correlate with the model's real-world performance. In the evaluation conducted in this work, the best models according to the training metrics do not correspond with the best models according to the test metrics. Consequently, it is crucial to evaluate these models with the test data (corresponding to actual trajectories from the online phase) using a proper metric such as the Euclidean distance.

5.2.2. Filtered dataset without empty values

In this section, we discuss the results obtained from training and testing the proposed models, including trilateration, using the filtered dataset without empty RSSI values, i.e., when $S_{\text{valid_min}}$=12. Additionally, we compare these results with the state-of-the-art best mean and median values reported in [15]. Table 4 lists these results, while Fig. 9 shows the error distribution for each model.

The analysis of this figure reveals how the absence of empty values affects each model. The baseline M1 notably improves in average error and dispersion compared with the results obtained when considering empty values (see Fig. 8). Models M5 and M6 benefit from lower maximum error values, maintaining similar average results.

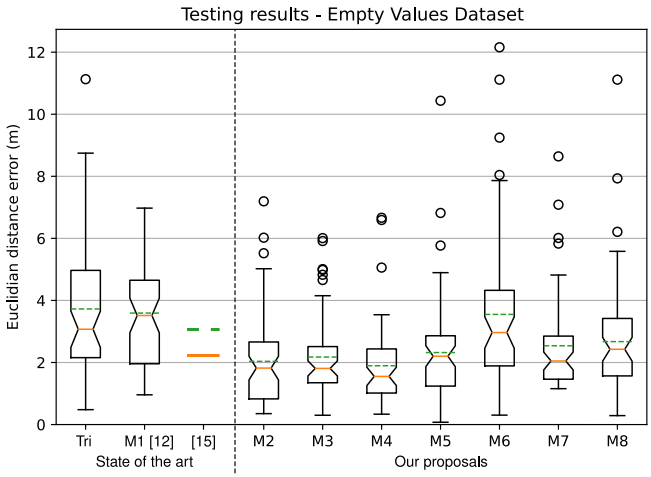

Fig. 8. Testing phase results are shown as the Euclidean distance error distribution between predicted and ground truth positions from the online fingerprint stage using the filtered dataset with empty values. In the case of [15], only the median and the mean results are provided. "Tri". represents the trilateration method.

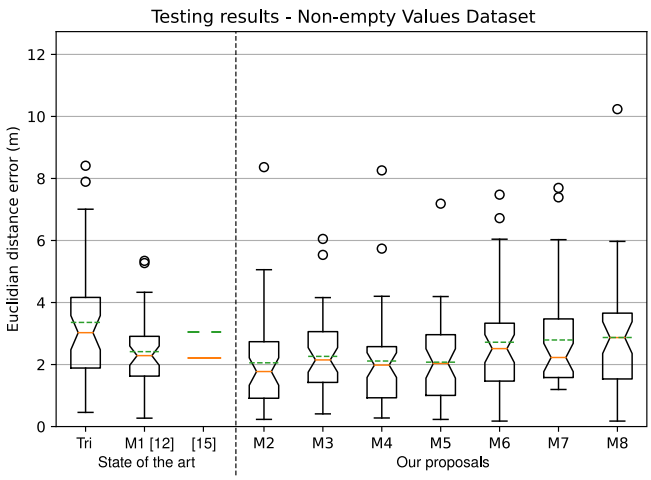

Fig. 9. Testing phase results are presented as the Euclidean distance error distribution between predicted and ground truth positions from the online fingerprint stage using the filtered dataset without empty values. In the case of [15], only the median and the mean results are provided. "Tri". represents the trilateration method.

Conversely, models with additional input data (M3 and M4) show a slight performance decrease, with increased average error and dispersion. This is expected as the additional input provides redundant information as there are no empty values.

When comparing M7, M6, and M8, the relative advantage of M7 decreases when empty values are not considered. Although M7 still exhibits the best median error of this group, its average and maximum errors closely align with those of the other models.

Even though M2, M4 and M5 closely compete for the top performance without empty values, M2 stands out for its stability across both filtered datasets, suggesting versatility and adaptability with relatively low average errors.

Furthermore, the baseline model M1 has improved enough to surpass the state-of-the-art mean error reported in [15]. Models M2, M3, M4, M5, and M7 outperform both the mean and median errors of the state-of-the-art. Although models M6 and M8 do not surpass the median errors of the state-of-the-art, they do exceed its mean error.

In summary, M4 is the best option when empty values are allowed, followed by M2 for its stability. If there are no empty values, models M5 and M2 are preferable. Furthermore, regression models are generally recommended over classification models in both scenarios.

6. Discussion

The performance of the proposed models strongly correlates with their design complexity. For instance, M2 outperforms M1, demonstrating that additional layers and neurons enable the network to capture deeper features from the RSSI signals. Both models

Table 6
Average prediction times (in milliseconds ± standard deviation) of neural network models across 100 executions per model.

Model	Prediction times (ms ± stddev)	
	With empty values	Without empty values
M1	0.9179 ± 0.0885	1.1797 ± 0.0571
M2	0.9031 ± 0.0439	1.1812 ± 0.0463
M3	1.1340 ± 0.0976	1.4305 ± 0.0623
M4	1.0936 ± 0.0398	1.5253 ± 0.1135
M5	1.0349 ± 0.0651	1.4872 ± 0.2178
M6	0.9606 ± 0.0552	1.2469 ± 0.0837
M7	0.9788 ± 0.0637	1.2691 ± 0.0879
M8	1.0352 ± 0.0547	1.3889 ± 0.1128

exhibit similar computational times (see Table 6), with M2 showing a lower deviation. This suggests that more complex models like M2 do not necessarily involve a higher computational load. Additionally, M1, which follows the guidelines provided by [12], only achieves acceptable performance in scenarios without empty values, highlighting the limitations of simpler architectures. Therefore, M2 stands out for its computational efficiency and adaptability, maintaining good performance in both tested scenarios while keeping prediction times low. These results make M2 highly suitable for real-time applications.

Models M3 and M4, which include an auxiliary mask to identify empty values, show better performance, especially in scenarios with empty values. However, as shown in Table 6, these models are computationally more demanding due to the processing of an additional input. The comparison between these two highlights how M4's more complex design, which processes inputs separately before combining them, captures signal features more effectively while requiring less computational time. These findings highlight the importance of a well-optimized design capable of handling missing or noisy data while capturing deeper features.

Finally, the comparison between regression models (M1–M5) and classification models (M6–M8) reveals higher accuracy for the former. The classification architectures, particularly M8, have room for improvement, as its dual-resolution grid fails to enhance accuracy and worsens performance. Future research could attempt to improve the effectiveness and efficiency of these models by exploring hybrid architectures that integrate both regression and classification approaches.

7. Conclusions

In this work, we explored the use of neural networks for BLE-based indoor positioning through the fingerprint technique. Our approach involved a novel proposal for the data capture and filtering of BLE signals, along with an analysis to identify the optimal parameters for this process. We have also proposed several neural network models following both regression and classification approaches and assessed their effectiveness through detailed comparisons.

We found that the baseline model (M1), which follows state-of-the-art recommendations, performed well only in the absence of missing values, while our proposed M2 model proved to be the most stable across different scenarios. Models M3 and M4, designed to handle missing values, showed superior performance in scenarios with noisy signals. Among the classification models, improvement was achieved by reducing the number of categories (the resolution of the grid). However, overall, the regression models demonstrated better adaptability and performance.

Moreover, our models M2, M3, M4, M5, and M7 not only outperformed the baseline model but also surpassed in both tests the best mean and median errors of the state-of-the-art results reported in [15] when using real RSSI measurements.

Our experiments also highlighted a key observation: training metrics do not always reflect a network's real-world performance. This emphasizes the need to evaluate models in practical applications, not just based on training metrics.

The findings of this study indicate that these models, particularly M2, have potential in practical applications. The stability of M2 across scenarios and its low computational time make it suitable for real-time indoor positioning applications in environments such as smart buildings and healthcare. For instance, M2 could be effectively deployed in patient monitoring systems, navigation within stores, museums, airports, or other large buildings, as well as in asset tracking within smart homes. The ability of M3 and M4 to handle noisy environments with missing data can be leveraged in challenging industrial or commercial settings where signal loss is common.

Overall, our study not only enhances the understanding of BLE positioning using neural network models for fingerprinting but also provides a solid foundation for future research.

Despite the good results obtained, we recognize limitations, such as dataset quality and environmental variability. As future work, we aim to build our own system for data collection, which will allow us to create a more heterogeneous dataset. This dataset will include various types of rooms, different room sizes, a greater number of capture points, and diverse environmental conditions, such as different types of objects within rooms and the presence of moving people. Furthermore, future efforts could explore more advanced techniques, consider other types of wireless signals such as WiFi and 5G, and implement these networks in various indoor positioning scenarios. Additionally, environment-independent models that can be easily applied in unknown locations could be studied.

CRediT authorship contribution statement

Alberto Ferrero-López: Writing – original draft, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Antonio Javier Gallego:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Investigation, Formal analysis, Conceptualization. **Miguel Angel Lozano:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Investigation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] D. Hill, A History of Engineering in Classical and Medieval Times, Routledge, London, 2013, http://dx.doi.org/10.4324/9781315800110.

[2] M. Jones, Tycho Brahe, Cartography and landscape in 16th century scandinavia, in: H. Palang, H. Sooväli, M. Antrop, G. Setten (Eds.), European Rural LandScapes: Persistence and Change in a Globalising Environment, Springer Netherlands, Dordrecht, 2004, pp. 209–226, http://dx.doi.org/10.1007/978-0-306-48512-1_13.

[3] B. Hofmann-Wellenhof, H. Lichtenegger, J. Collins, Global Positioning System, Springer, Vienna, 2001, http://dx.doi.org/10.1007/978-3-7091-6199-9.

[4] C. Alexandrow, in: Defense Advanced Research Projects Agency (DARPA) (Ed.), The Story of GPS, Vol. 50, DARPA, 2008.

[5] S.J. Hayward, K. van Lopik, C. Hinde, A.A. West, A survey of indoor location technologies, techniques and applications in industry, Internet Things (2022) 100608, Publisher: Elsevier.

[6] G. Retscher, Indoor navigation—User requirements, state-of-the-art and developments for smartphone localization, Geomatics 3 (1) (2023) 1–46, publisher: Multidisciplinary Digital Publishing Institute.

[7] G. Dedes, A. Dempster, Indoor GPS positioning - challenges and opportunities, in: VTC-2005-Fall. 2005 IEEE 62nd Vehicular Technology Conference, 2005, Vol. 1, (ISSN: 1090-3038) 2005, pp. 412–415, http://dx.doi.org/10.1109/VETECF.2005.1557943.

[8] T. Yang, A. Cabani, H. Chafouk, A survey of recent indoor localization scenarios and methodologies, Sensors 21 (23) (2021) 8086, http://dx.doi.org/10.3390/s21238086, number: 23 Publisher: Multidisciplinary Digital Publishing Institute.

[9] M. Sauter, From GSM To LTE: An Introduction To Mobile Networks and Mobile Broadband, Books, Wiley Online Library, 2010, URL https://books.google.es/books?id=uso-6LN2YjsC.

[10] F.S. Daniş, A.T. Cemgil, Model-based localization and tracking using bluetooth low-energy beacons, Sensors 17 (11) (2017) 2484, http://dx.doi.org/10.3390/s17112484, number: 11 Publisher: Multidisciplinary Digital Publishing Institute.

[11] R. Faragher, R. Harle, Location fingerprinting with bluetooth low energy beacons, IEEE J. Sel. Areas Commun. 33 (11) (2015) 2418–2428, http://dx.doi.org/10.1109/JSAC.2015.2430281.

[12] A. Puckdeevongs, Indoor localization using RSSI and artificial neural network, in: 2021 9th International Electrical Engineering Congress, iEECON, 2021, pp. 479–482, http://dx.doi.org/10.1109/iEECON51072.2021.9440373.

[13] R. Joseph, S.B. Sasi, Indoor positioning using WiFi fingerprint, in: 2018 International Conference on Circuits and Systems in Digital Enterprise Technology, ICCSDET, 2018, pp. 1–3, http://dx.doi.org/10.1109/ICCSDET.2018.8821184.

[14] A.B. Adege, L. Yen, H.-p. Lin, Y. Yayeh, Y.R. Li, S.-S. Jeng, G. Berie, Applying deep neural network (DNN) for large-scale indoor localization using feed-forward neural network (FFNN) algorithm, in: 2018 IEEE International Conference on Applied System Invention, ICASI, 2018, pp. 814–817, http://dx.doi.org/10.1109/ICASI.2018.8394387.

[15] F.S. Daníş, A.T. Cemgíl, C. Ersoy, Adaptive sequential Monte Carlo filter for indoor positioning and tracking with bluetooth low energy beacons, IEEE Access 9 (2021) 37022–37038, http://dx.doi.org/10.1109/ACCESS.2021.3062818.

[16] X. Wang, L. Gao, S. Mao, S. Pandey, CSI-based fingerprinting for indoor localization: A deep learning approach, IEEE Trans. Veh. Technol. 66 (1) (2017) 763–776, http://dx.doi.org/10.1109/TVT.2016.2545523.

[17] J. Torres-Sospedra, R. Montoliu, A. Martínez-Usó, J.P. Avariento, T.J. Arnau, M. Benedito-Bordonau, J. Huerta, UjiIndoorLoc: A new multi-building and multi-floor database for WLAN fingerprint-based indoor localization problems, in: 2014 International Conference on Indoor Positioning and Indoor Navigation, IPIN, 2014, pp. 261–270, http://dx.doi.org/10.1109/IPIN.2014.7275492.

[18] K. Szyc, M. Nikodem, M. Zdunek, Bluetooth low energy indoor localization for large industrial areas and limited infrastructure, Ad Hoc Netw. 139 (2023) 103024, http://dx.doi.org/10.1016/j.adhoc.2022.103024.

[19] G. Li, E. Geng, Z. Ye, Y. Xu, J. Lin, Y. Pang, Indoor positioning algorithm based on the improved RSSI distance model, Sensors 18 (9) (2018) 2820, http://dx.doi.org/10.3390/s18092820.

[20] F.S. Daniş, A.T. Naskali, A.T. Cemgil, C. Ersoy, Position-annotated-BLE-RSSI-dataset, 2021, http://dx.doi.org/10.34740/KAGGLE/DS/1662453, URL https://www.kaggle.com/ds/1662453.

[21] F.S. Daniş, A.T. Naskali, A.T. Cemgil, C. Ersoy, An indoor localization dataset and data collection framework with high precision position annotation, Pervasive Mob. Comput. 81 (2022) 101554, http://dx.doi.org/10.1016/j.pmcj.2022.101554.

[22] N. Singh, S. Choe, R. Punmiya, Machine learning based indoor localization using wi-fi RSSI fingerprints: An overview, IEEE Access 9 (2021) 127150–127174, http://dx.doi.org/10.1109/ACCESS.2021.3111083.

[23] H. Jin, F. Chollet, Q. Song, X. Hu, AutoKeras: An automl library for deep learning, J. Mach. Learn. Res. 24 (6) (2023) 1–6.

[24] T. O'Malley, E. Bursztein, J. Long, F. Chollet, H. Jin, L. Invernizzi, et al., Kerastuner, 2019, https://github.com/keras-team/keras-tuner.